PERTEMUAN 1 · Teori terapan

Pengenalan Kompilasi dan Arsitektur Kompilator

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 01 — Menjelaskan tahapan kompilasi (*lexical*, *syntax*, *semantic analysis*).

Sub-CPMK yang dinilai pada pertemuan ini

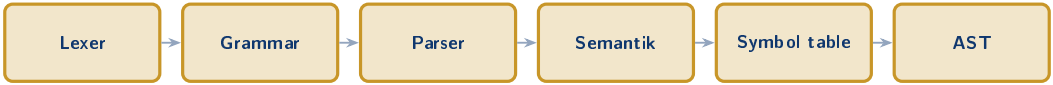
- **Sub-CPMK 1.1:** Menjelaskan perbedaan interpreter dan compiler serta peran assembler, interpreter, dan compiler sebagai translator, dengan menyebut contoh masukan dan keluaran masing-masing.
- **Sub-CPMK 1.2:** Menjelaskan fase-fase kompilator (analisis leksikal, sintaksis, semantik, dan pembangkitan kode antara) beserta peran tabel simbol dan penanganan kesalahan.
- **Sub-CPMK 1.3:** Membuat tabel penelusuran fase kompilasi untuk satu program C sederhana, yang memuat keluaran antara berupa token, pohon sintaks, dan kode antara.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
10	Pembuka, orientasi capaian, dan peta proyek mini-compiler
30	Keluarga penerjemah; kompilator dibandingkan interpreter
35	Fase analisis dan sintesis; tabel simbol dan penanganan kesalahan
30	Penelusuran satu program C melalui token, pohon, dan kode antara
30	Latihan terstruktur: mengisi tabel penelusuran fase
15	Diskusi hasil latihan, refleksi, dan penugasan

Pertemuan teori terapan: ceramah singkat diikuti latihan terstruktur dan pemeriksaan bersama.

Posisi Pertemuan pada Rantai Proyek



- Proyek semester: *front-end* mini-compiler untuk subset bahasa C, dibangun bertahap.
- Pertemuan ini memberi peta seluruh rantai: lexer, grammar, parser, semantik, *symbol table*, dan AST.
- Setiap pertemuan berikutnya menambah satu simpul rantai itu.

Milestone M0

Menyiapkan repositori, spesifikasi Subset-C v0.1, dan program contoh sebagai fondasi Praktikum 1.

Keluarga Penerjemah

Penerjemah (*translator*)

Program yang mengubah program sumber ke bahasa lain; keluarannya dapat berupa assembly, kode mesin, *bytecode*, atau bahasa tingkat tinggi lain.

Jenis	Masukan	Keluaran
Assembler	Bahasa assembly	Kode mesin
Kompilator	Bahasa tingkat tinggi	Assembly atau kode mesin
Interpreter	Program sumber	Aksi program, tanpa kode target tetap
Cross-compiler	Program untuk platform lain	Kode untuk platform target

Kompilator dan interpreter sama-sama membaca program sumber, tetapi model eksekusinya berbeda.

Kompilator dibandingkan Interpreter

Aspek	Kompilator	Interpreter
Keluaran	Kode target yang berdiri sendiri	Tidak ada kode target permanen
Waktu eksekusi	Cepat setelah kompilasi	Lebih lambat, diterjemahkan saat jalan
Waktu mulai	Lebih lama karena kompilasi	Lebih cepat
Portabilitas	Kode target terikat platform	Selama interpreter tersedia
Analisis	Menyeluruh sebelum eksekusi	Terbatas pada saat eksekusi
Contoh	GCC, Clang, javac	Python, Ruby, JavaScript

Batas yang tidak selalu tegas

Java mengompilasi sumber ke *bytecode*, lalu mesin virtual menjalankannya dan dapat melakukan kompilasi *just-in-time*.

Cross-Compiler dan Bootstrapping

Cross-compiler

Berjalan di satu platform, menghasilkan kode untuk platform lain. Contoh: kompilasi untuk mikrokontroler dari komputer desktop.

- wajib tahu konvensi pemanggilan;
- ukuran kata dan tata letak memori target;
- ketersediaan pustaka target.

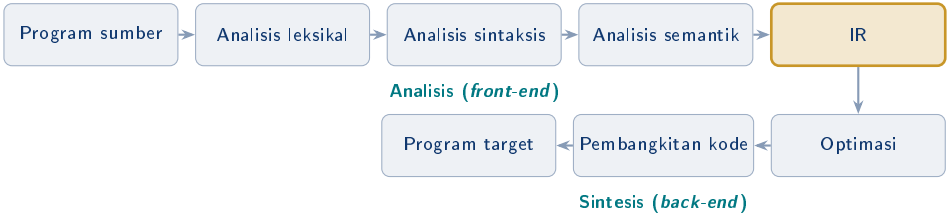
Bootstrapping

Membangun kompilator suatu bahasa dengan kompilator bahasa itu sendiri.

- dimulai dari kompilator sederhana dalam bahasa lain;
- ditingkatkan bertahap sampai mandiri;
- GCC dan Clang memiliki sejarah bootstrapping panjang.

Istilah *compiler* berasal dari gagasan mengompilasi kumpulan rutin menjadi satu program; kini kompilator juga menganalisis, mengoptimasi, dan membangkitkan kode.

Fase Analisis dan Sintesis



- Analisis memahami struktur program sumber.
- Sintesis menghasilkan program target.
- *Intermediate representation* (IR) menjadi batas keduanya.

Analisis Leksikal

- Memindai karakter demi karakter dari program sumber.
- Mengelompokkan karakter menjadi token: jenis dan atribut.
- Membuang spasi, baris baru, dan komentar, tetapi menyimpan lokasi untuk diagnostik.
- Keluaran: aliran token yang siap diproses parser.

Contoh

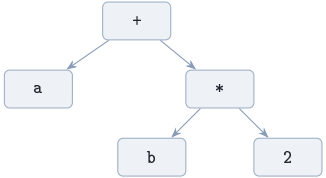
`int a = 5;` menghasilkan aliran `INT`, `ID(a)`, `ASSIGN`, `INT_LIT(5)`, `SEMI`.

Token `INT_LIT` membawa atribut nilai 5; token `ID` membawa nama `a`.

Analisis Sintaksis

- Memeriksa apakah aliran token sesuai tata bahasa.
- Menyusun token menjadi pohon urai atau AST.
- Parser dapat mengisi tabel simbol awal.
- Kesalahan sintaksis dilaporkan dan dipulihkan.

Keluaran fase ini adalah pohon sintaks yang hubungan induk–anaknya sesuai aturan tata bahasa.



Pohon untuk `a + b * 2`

Analisis Semantik

- Memeriksa makna program, bukan sekadar bentuknya.
- Pertanyaan yang dijawab: apakah identifier sudah dideklarasikan, apakah tipe operand cocok, apakah fungsi dipanggil dengan jumlah argumen yang benar.
- Menambahkan anotasi tipe dan referensi simbol pada AST.
- Keluaran: AST teranotasi yang siap diterjemahkan ke kode antara.

Contoh anotasi

`a : int`, `b : int`, `a + b : int`; hasil `print` bertipe `void`; `return 0` bertipe `int`.

Sintesis: Optimasi dan Pembangkitan Kode

Optimasi

- lokal: dalam satu blok dasar;
- global: lintas blok;
- masukan dan keluaran berupa kode antara yang lebih efisien.

Pembangkitan kode

- memilih instruksi target;
- mengalokasikan register;
- menghasilkan assembly atau kode mesin.

IR memisahkan pekerjaan memahami program (analisis) dari pekerjaan menghasilkan program target (sintesis), sehingga kedua bagian dapat dikembangkan terpisah.

Masukan dan Keluaran Setiap Fase

Fase	Masukan	Keluaran
Leksikal	Karakter sumber	Aliran token
Sintaksis	Aliran token	Pohon urai atau AST
Semantik	AST dan tabel simbol	AST teranotasi
IR	AST teranotasi	Kode antara
Optimasi	Kode antara	Kode antara yang lebih efisien
Pembangkitan kode	Kode antara	Assembly atau kode mesin

Setiap fase mempertahankan atau memperkaya informasi dari fase sebelumnya.

Tabel Simbol

Tabel simbol (*symbol table*)

Struktur data yang memetakan identifier ke tipe, scope, alamat memori, dan kategori (variabel, fungsi, parameter).

- Operasi utama: `insert`, `lookup`, `delete`.
- Scope bersarang diimplementasikan sebagai stack tabel atau tabel dengan rantai scope.
- Informasi yang disimpan: nama, tipe, ukuran, alamat, dan rujukan ke deklarasi.

Tabel simbol dipakai parser, analisis semantik, dan pembangkitan kode; karena itu disebut pusat informasi kompilator.

Jenis Kesalahan dan Pesan yang Informatif

Jenis	Contoh	Pesan informatif
Leksikal	karakter `@` tidak dikenal	baris 4, kolom 12: karakter `@` tidak dikenal
Sintaksis	titik koma hilang	baris 7: diharapkan `;` setelah ekspresi
Semantik	tipe tidak cocok	baris 9: tidak dapat menetapkan `float` ke `int` tanpa *cast*
Run-time	pembagian nol	baris 11: pembagian dengan nol saat eksekusi

Pemulihan kesalahan

Kompilator yang baik tidak berhenti pada kesalahan pertama. Strategi umum: *panic mode*, *phrase-level recovery*, dan *error production*. Pesan memuat lokasi, penyebab, dan saran perbaikan.

Contoh: Scope Bersarang pada Tabel Simbol

```
int x = 1;
{
    int x = 2;
    print(x);
}
print(x);
```

Listing 1: Dua deklarasi x pada scope berbeda

1. Masuk blok: tabel scope dalam didorong ke stack.
2. `int x = 2` menutupi (*shadow*) `x` luar.
3. `print(x)` di dalam blok memakai `x = 2`.
4. Blok selesai: tabel scope dalam dibuang.
5. `print(x)` terakhir memakai `x = 1`.

Perilaku ini diimplementasikan pada Pertemuan 14 (Bab XV).

Program Uji dan Aliran Tokennya

```
// Program Subset-C pertama:
    penjumlahan dua bilangan
int main() {
    int a = 5;
    int b = 7;
    int hasil = a + b;
    print(hasil);
    return 0;
}
```

Listing 2: Program `hello.sc` dari proyek Subset-C

Aliran token

`INT · ID(main) · LPAREN · RPAREN · LBRACE · INT · ID(a) · ASSIGN · INT_LIT(5) · SEMI · INT · ID(b) · ASSIGN · INT_LIT(7) · SEMI · INT · ID(hasil) · ASSIGN · ID(a) · PLUS · ID(b) · SEMI · PRINT · LPAREN · ID(hasil) · RPAREN · SEMI · RETURN · INT_LIT(0) · SEMI · RBRACE · EOF`

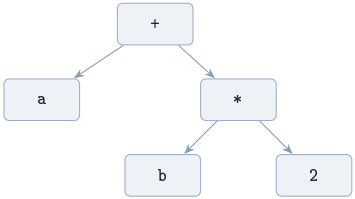
Setiap token membawa jenis, lexeme, dan lokasi (baris, kolom).

Tabel Penelusuran Fase untuk `hello.sc`

Fase	Keluaran untuk program contoh
Leksikal	34 token berurutan: `INT ID(main) LPAREN RPAREN LBRACE INT ID(a) ASSIGN INT_LIT(5) SEMI ... RBRACE EOF`
Sintaksis	`Program` → `DeklarasiFungsi(main)` → `Blok`: deklarasi `a`, `b`, `hasil`; pernyataan `print(hasil)`; pernyataan `return 0`
Semantik	`a:int`, `b:int`, `hasil:int`; `a + b:int`; `print` menerima satu argumen bertipe `int`; hasil `print` bertipe `void`
Kode antara	`t1 = a + b`; `hasil = t1`; `param hasil`; `call print, 1`; `t2 = 0`; `return t2`

Setiap sel merujuk baris program sumber, bukan istilah umum. Tabel seperti ini menjadi artefak Sub-CPMK 1.3.

Contoh Berpenyelesaian: Pohon ke Kode Antara



Pohon sintaks `a + b * 2`

Kode antara tiga alamat

```
t1 = b * 2
t2 = a + t1
```

- Perkalian lebih dalam pada pohon, jadi dikerjakan lebih dulu.
- `t1` dan `t2` adalah variabel sementara.
- Urutan instruksi mencerminkan urutan evaluasi pohon.

Kegiatan Mahasiswa di Kelas

Latihan terstruktur: tabel penelusuran fase (Sub-CPMK 1.3)

1. Pilih satu program C maksimal sepuluh baris, misalnya dari `examples/`.
2. Tuliskan aliran token beserta baris dan kolomnya.
3. Gambarkan pohon sintaks untuk program tersebut.
4. Tuliskan kode antara tiga alamat.
5. Isi tabel dengan kolom fase, masukan, keluaran, dan bukti.
6. Tukar tabel dengan teman dan periksa satu sama lain.

Berkas uji: `examples/hello.sc`, `examples/arith.sc`.

Luaran: tabel penelusuran fase yang dapat diperiksa baris per baris.

Tugas dan Artefak

Artefak Sub-CPMK 1.3

Tabel penelusuran fase untuk satu program C sederhana, memuat keluaran antara berupa **token**, **pohon sintaks**, dan **kode antara**.

- Program maksimal sepuluh baris; setiap sel merujuk baris program sumber.
- Pohon sintaks konsisten dengan aliran token.
- Kode antara konsisten dengan pohon sintaks.

Tambahan proyek M0

Repositori dengan struktur `spec`, `include`, `src`, `examples`, `tests`; spesifikasi v0.1 terbaca; minimal tiga program contoh; satu halaman catatan keputusan desain.

M0 menjadi prasyarat Praktikum 1; tabel penelusuran menjadi bukti Sub-CPMK 1.3 pada UTS Praktik.

Rangkuman dan Jembatan

- Penerjemah mencakup assembler, kompilator, interpreter, dan *cross-compiler*; masing-masing punya masukan dan keluaran berbeda.
- Kompilator bekerja dalam dua bagian: analisis (front-end) dan sintesis (back-end), dipisahkan oleh IR.
- Fase leksikal menghasilkan token, sintaksis menghasilkan pohon, semantik menambahkan tipe, dan IR menghasilkan kode antara.
- Tabel simbol adalah pusat informasi; pesan kesalahan yang baik menyebut lokasi dan penyebab.
- Tabel penelusuran fase membuktikan bahwa setiap keluaran dapat dilacak ke program sumber.

Pertemuan berikutnya

Pertemuan 2 memperdalam landasan formal: teori bahasa, grammar, dan hierarki Chomsky. Landasan ini menjelaskan mengapa lexer cukup memakai otomata hingga, sedangkan parser memerlukan otomata pushdown.